

# LAB 3: ARCHITECTURE SELECTION & COMPONENT MODELLING

## Community Solar Credit Allocation Manager

### 1. Scenario Review

**Functional requirements:** register solar generation; view available solar credits; allocate credits to eligible residents; validate allocations so they do not exceed available credits; view credit balance; view allocation history; and notify residents when allocations are updated.

**Non-functional requirements:** credit balance should be displayed within 3 seconds for at least 95% of tested requests, and credit-allocation management should be restricted to authorized Co-op Managers.

**Main challenges:** performance for balance retrieval, usability for different user roles, and security/authorization for credit allocation.

### 2. Architectural Style Analysis

**Layered Architecture:** organizes the system into horizontal layers such as Presentation, Business Logic, and Data. It provides separation of concerns and is easy to understand, but may introduce layer-to-layer overhead and can make independent scaling of individual components more difficult.

**Microservices Architecture:** organizes the system as small, independently deployable services. It supports independent scaling and fault isolation, but introduces operational complexity, network latency, and data-consistency challenges.

**Client-Server Architecture:** uses a centralized server with multiple clients. It provides centralized control and simple deployment, but can create a single point of failure and a scalability bottleneck.

### 3. Architecture Selection & Justification

We chose Layered Architecture for the Community Solar Credit Allocation Manager.

**Reason 1 — Clear separation of responsibilities:** Prosumer Residents, Residents, and Co-op Managers perform different operations. The Presentation layer can handle their interactions, the Business Logic layer can enforce credit-allocation and validation rules, and the Data layer can manage persistent records. This separation supports maintainability and clear responsibility boundaries.

**Reason 2 — Fits the defined workflow:** the system follows a structured flow in which generation is recorded, available credits are determined, an allocation is requested, the allocation is validated, and resident credit/history information is updated. Layered Architecture keeps these business rules separate from the interface and data-storage concerns.

**Security advantage:** authorization can be enforced in the Business Logic layer before sensitive allocation operations are executed. This supports the requirement that only authorized Co-op Managers manage credit allocations.

**Performance benefit:** the separation of data access and business processing allows frequently used operations such as credit-balance retrieval to be optimized independently, supporting the stated 3-second/95% performance requirement.

### 4. Component Identification

1. **User Interface Component** — handles interactions for Prosumer Residents, Residents, and Co-op Managers.
2. **Solar Generation Management Component** — records and manages rooftop solar generation.
3. **Credit Allocation & Validation Component** — processes allocations and prevents requests from exceeding available credits.
4. **Credit & Billing Tracking Component** — manages balances, allocation history, and billing-offset information.
5. **Notification Service Component** — sends allocation-update notifications.
6. **Solar Credit Database Component** — stores generation, credit, history, and billing records.

### 5. Key Interfaces and Dependencies

The component diagram shows service/API interactions between the User Interface and the business components, an available-credit check between Credit Allocation & Validation and Solar Generation Management, an update interaction with Credit & Billing Tracking, a notification trigger, and database read/write interactions. Provided interfaces are represented with ball notation and required interfaces with socket notation where applicable.

### 6. Diagram Finalization & Export

Before submission, verify that all six components are present, interface directions and labels are correct, and UML component notation is used consistently. Export the Component Diagram as PNG or PDF, save the written justification as PDF, and submit the required files under the Lab3 folder in the SELABS GitHub repository.